INTRODUCTION TO CLASS

Objectives

- Object and Classes
- Constructors in Java
- Static Modifier

INTRODUCTION TO CLASS

Object and Classes

Since Java is an object oriented language, complete java language is build on classes and object. Java is also known as a strong **Object oriented programming language**(oops).

OOPS is a programming approach which provides solution to problems with the help of algorithms based on real world. It uses real world approach to solve a problem. So object oriented technique offers better and easy way to write program then procedural programming model such as C, ALGOL, PASCAL.

Main Features of OOPS

- Inheritence
- Polymorphism
- Encapsulation
- Abstraction

As an object oriented language Java supports all the features given above. We will discuss all these features in detail later.

Class

In Java everything is encapsulated under classes. Class is the core of Java language. Class can be defined as a template/ blueprint that describe the behaviors /states of a particular entity. A class defines new data type. Once defined this new type can be used to create object of that type. Object is an instance of class. You may also call it as physical existence of a logical template class.

A class is declared using **class** keyword. A class contains both data and code that operate on that data. The data or variables defined within a **class** are called **instance variables** and the code that operates on this data is known as **methods**.

Rules for Java Class

- A class can have only public or default(no modifier) access specifier.
- It can be either abstract, final or concrete (normal class).
- It must have the class keyword, and class must be followed by a legal identifier.
- It may optionally extend one parent class. By default, it will extend java.lang.Object.
- It may optionally implement any number of comma-separated interfaces.
- The class's variables and methods are declared within a set of curly braces {}.

Rules for Java Class

- Each **.java** source file may contain only one public class. A source file may contain any number of default visible classes.
- Finally, the source file name must match the public class name and it must have a .java suffix.

A simple class example

Suppose, Student is a **class** and student's name, roll number, age will be its property. Lets see this in Java syntax

```
class Student
{
 String name;
 int rollno;
 int age;
}
```

Rules for Java Class

When a reference is made to a particular student with its property then it becomes an **object**, physical existence of Student class.

Student std=new Student();

After the above statement **std** is instance/object of Student class. Here the **new** keyword creates an actual physical copy of the object and assign it to the **std** variable. It will have physical existence and get memory in heap area. **The new operator dynamically allocates memory for an object**

Objects in Java:

Let us now look deep into what are objects. If we consider the real-world we can find many objects around us, Cars, Dogs, Humans, etc. All these objects have a state and behavior.

If we consider a dog, then its state is - name, breed, color, and the behavior is - barking, wagging, running

If you compare the software object with a real world object, they have very similar characteristics.

Software objects also have a state and behavior. A software object's state is stored in fields and behavior is shown via methods.
So in software development, methods operate on the internal state of an object and the object-to-object communication is done via methods.

Creating an Object:

As mentioned previously, a class provides the blueprints for objects. So basically an object is created from a class. In Java, the new key word is used to create new objects.

There are three steps when creating an object from a class:

- **Declaration:** A variable declaration with a variable name with an object type.
- **Instantiation:** The 'new' key word is used to create the object.
- **Initialization:** The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

Creating an Object:

Example of creating an object is given below:

```
public class Puppy{
public Puppy(String name){
// This constructor has one parameter, name.
System.out.println("Passed Name is :"+ name );
}
Public static void main(String[]args){
// Following statement would create an object myPuppy
Puppy myPuppy =newPuppy("tommy");
}
}
```

If we compile and run the above program, then it would produce the following result:

PassedNameis:tommy

Accessing Instance Variables and Methods:

Instance variables and methods are accessed via created objects. To access an instance variable the fully qualified path should be as follows:

```
/* First create an object */
ObjectReference=newConstructor();

/* Now call a variable as follows */
ObjectReference.variableName;

/* Now you can call a class method as follows */
ObjectReference.MethodName();
```

Accessing Instance Variables and Methods:

Example:

This example explains how to access instance variables and methods of a class:

```
public class Puppy{

int puppy Age;

public Puppy(String name){
// This constructor has one parameter, name.
System.out.println("Name chosen is :"+ name );
}
```

Accessing Instance Variables and Methods:

```
public void setAge(int age ){
      puppyAge = age;
}

public int getAge(){
System.out.println("Puppy's age is :"+ puppyAge );
return puppyAge;
}

public static void main(String[]args){
/* Object creation */
Puppy myPuppy =newPuppy("tommy");
```

Accessing Instance Variables and Methods:

```
/* Call class method to set puppy's age */
    myPuppy.setAge(2);

/* Call another class method to get puppy's age */
    myPuppy.getAge();

/* You can access instance variable as follows as well */
System.out.println("Variable Value :"+ myPuppy.puppyAge );
}
}
```

If we compile and run the above program, then it would produce the following result:

Accessing Instance Variables and Methods:

Name chosen
is:tommy Puppy's age
is :2 Variable Value :2

Constructors in Java

A constructor is a special method that is used to initialize an object.Every class has a constructor,if we don't explicitly declare a constructor for any java class the compiler builds a default constructor for that class. A constructor does not have any return type.

A constructor has same name as the class in which it resides. Constructor in Java cannot be abstract, static, final or synchronized. These modifiers are not allowed for constructor.

```
class Car
{
 String name ;
 String model;
```

Constructors in Java

```
Car( )    //Constructor
 {
  name ="";
  model="";
 }
}
```

There are two types of Constructor

- Default Constructor
- Parameterized constructor

Each time a new object is created at least one constructor will be invoked.

```
Car c = new Car()        //Default constructor invoked
Car c = new Car(name);   //Parameterized constructor invoked
```

Constructor Overloading

Like methods, a constructor can also be overloaded. Overloaded constructors are differentiated on the basis of their type of parameters or number of parameters. Constructor overloading is not much different than method overloading. In case of method overloading you have multiple methods with same name but different signature, whereas in Constructor overloading you have multiple constructor with different signature but only difference is that Constructor doesn't have return type in Java.

Example of constructor overloading

```
class Cricketer
{
 String  name;
 String   team;
 int age;
 Cricketer ()   //default constructor.
 {
  name ="";
  team ="";
  age = 0;
 }
```

Example of constructor overloading

```
Cricketer(String n, String t, int a)  //constructor overloaded
{
 name = n;
 team = t;
 age = a;
}
Cricketer (Cricketer ckt)          //constructor similar to copy
constructor of c++
{
 name = ckt.name;
 team = ckt.team;
 age = ckt.age;
}
```

Example of constructor overloading

```
 public String toString()
 {
  return "this is " + name + " of "+team;
 }
}

Class test:
{
 public static void main (String[] args)
 {
  Cricketer c1 = new Cricketer();
  Cricketer c2 = new Cricketer("sachin", "India", 32);
```

Example of constructor overloading

```
  Cricketer c3 = new Cricketer(c2 );
  System.out.println(c2);
  System.out.println(c3);
  c1.name = "Virat";
  c1.team= "India";
  c1.age = 32;
  System .out. print in (c1);
 }
}
```

output:

this is sachin of

india this is sachin

of india